1. $(\varepsilon, 000, start) \vdash_M$
   $(X, 00, find0) \vdash_M$
   $(\varepsilon, XX0, reset) \vdash_M$
   $(\varepsilon, \square XX0, reset) \vdash_M$
   $(\varepsilon, XX0, start) \vdash_M$
   $(X, X0, start) \vdash_M$
   $(XX, 0, start) \vdash_M$
   $(XXX, \square, find0) \vdash_M$
   $(XXX, \square, N)$

2. Vom face reducerea $\texttt{HALT} \leq f$; știm de la curs că $\texttt{HALT} \notin R$.

$$\texttt{HALT} \leq_m f \overset{\text{def}}{=} \exists t : \Sigma^* \to \Sigma^*, \text{ a.î. } \begin{cases} t \text{ e computabilă} \\ \forall (M, w), \quad \texttt{HALT}(enc((M, w))) = TRUE \Leftrightarrow f(t(enc((M, w)))) = TRUE \end{cases}$$

   Transformarea t produce mașina M':

$M'[v]$:
   1. run $M[w]$
   2. accept

   t trebuie doar să producă definiția mașinii, deci e computabilă.

   Demonstrăm echivalența, cu două demonstrații:

   "$\Rightarrow$": $\texttt{HALT}(enc((M, w))) = TRUE \Rightarrow M[w]$ halts $\Rightarrow M'$ acceptă toate cuvintele $\Rightarrow f(enc(M')) = TRUE$.

   "$\Leftarrow$": $f(enc(M')) = TRUE \Rightarrow M'$ acceptă mai multe cuvinte decât respinge $\Rightarrow$
   nu poate să accepte și să respingă zero cuvinte $\Rightarrow M[w]$ halts $\Rightarrow \texttt{HALT}(enc(M, w)) = TRUE$.

3. O să demonstrăm explicit că $T(n) \in O(n^{\log_3 4} - kn)$, pentru $k \in \mathbb{Z}$; deoarece termenul $-kn$ crește strict mai încet ca $n^{\log_3 4}$, această mulțime e egală cu $O(n^{\log_3 4})$.

   Expandăm definiția lui O pentru a explicita ce înseamnă $T(n) \in O(n^{\log_3 4} - kn)$:

$$\exists c \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0 : \quad 0 \leq T(n) \leq c(n^{\log_3 4} - kn)$$

   Demonstrăm inegalitățile prin inducție:

   Caz de bază ($n = 1$):

$$0 \leq T(1) \leq c(1 - k) \tag{1}$$
$$0 \leq 1 \leq c(1 - k) \tag{2}$$

   Propoziție adevărată, dacă $c \geq \frac{1}{1-k}$ (ca relația să fie posibilă, trebuie ca $1 - k > 0$).

   Pas inductiv Presupunem $0 \leq T(\frac{n}{3}) \leq c((\frac{n}{2})^{\log_3 4} - k\frac{n}{3})$
   Trebuie să arătăm că: $0 \leq T(n) \leq c(n^{\log_3 4} - kn)$
   Pentru inegalitățile din ipoteza inductivă, înmulțim cu 4 și adunăm n; obținem:

$$n \leq T(n) \leq cn^2 - 2ckn + n \tag{3}$$
$$= cn^2 - ckn + n - ckn \tag{4}$$
$$\leq c(n^2 - kn) \tag{5}$$

   $\forall n \geq 1, \quad n \geq 0$, deci inegalitatea $0 \leq T(n)$ e adevărată.
   Ultima inegalitate ține doar dacă $(1 - ck)n \leq 0$; adică $c \geq \frac{1}{k}$.
   Alegând $k = \frac{1}{2}$ toate relațiile sunt valide și avem 2 constrângeri pe c; alegem $c = 2$.

4. Expandăm definiția lui O pentru a explicita ce înseamnă $2^n n \in O(3^n)$:

$$\exists c \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ a.î. } \forall n \geq n_0 : \quad 0 \leq 2^n n \leq c3^n$$

Demonstrăm prin inducție după n:
Caz de bază $(n = 1) : 0 \leq 2 \leq 3c$.
Adevărat pentru $c \geq \frac{2}{3}$.
Pas inductiv: Presupunem $0 \leq 2^{n-1}(n-1) \leq c3^{n-1}$
Pentru inegalitățile din ipoteza inductivă, înmulțim cu 2 și adunăm 2^n:

$$\begin{aligned} 2^n \leq 2^n n &\leq 2c3^{n-1} + 2^n && (6)\\ &= c3^n - c3^{n-1} + 2^n && (7)\\ &\leq c3^n && (8) \end{aligned}$$

Ultima inegalitate ține doar dacă $2^n - c3^{n-1} \leq 0$. Pentru $c = 3$, avem $2^n - 3^n \leq 0$

5. Bucla interioară se repetă de $\frac{n}{2}$ ori, deci are complexitate $\Theta(n)$.
Funcția e recursivă cu relația de recurență:

$$T(n) = T\left(\frac{n}{2}\right) + \frac{n}{2}$$

Identificăm $a = 1, b = 2, f(n) = \frac{n}{2}$; observăm $k = 1$, pentru care $\frac{n}{4} \leq k\frac{n}{4}$.
Condiția de regularitate ține, deci putem aplica cazul 3 al Teoremei Master.
Deci $T(n) \in \Theta(n)$.

6. $\texttt{SAT} \leq_P f \quad (1)$
$g \leq_P \texttt{SAT} \quad (2)$
$\exists h \in \mathbf{NP}, \quad (f \leq_P h \wedge h \leq_P g) \quad (3)$
$(1) \Rightarrow f \in \mathbf{NPH} \quad (4)$
$(2) \Rightarrow g \in \mathbf{NP} \quad (5)$
$(3) \Rightarrow f \in \mathbf{NP} \quad (6)$
$(4) \wedge (6) \Rightarrow f \in \mathbf{NPC} \quad (7)$
$(3) \wedge (7) \Rightarrow h \in \mathbf{NPC} \quad (8)$
$(3) \wedge (8) \Rightarrow g \in \mathbf{NPC}$

7. Programul încearcă să pună elementul pe prima poziție goală; dacă primele $\log_2 n$ poziții sunt pline, le va face 0 și va pune suma lor pe prima poziție liberă.

- calculăm costul:
- op1: Modifică doar A[0]. Cost = 1.
- op2: Modifică A[0] și A[1]. Cost = 2.
- op3: Modifică doar A[0]. Cost = 1.
- op4: Modifică A[0], A[1] și A[2]. Cost = 3.
- op5: Modifică doar A[0]. Cost = 1.
- op6: Modifică A[0] și A[1]. Cost = 2.
- op7: Modifică doar A[0]. Cost = 1.
- op8: Modifică A[0], A[1], A[2] și A[3]. Cost = 4.
- ...

Observăm un tipar operațiile care sunt puteri ale lui 2 (adică 2, 4, 8, 16, ...) vor avea un cost mai mare, deoarece resetează mai multe elemente ale vectorului.

$$\begin{aligned} \text{cost}(S) &= n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots \\ &= n \times \left(\left(\frac{1}{2} \right)^{\log_2(n)} - 1 \right) \div \left(\frac{1}{2} - 1 \right) \leq 2 \times n \\ \Rightarrow \text{cost}(op) &= \frac{\text{cost}(S)}{n} \leq 2 \\ &\Rightarrow \text{cost(op)} = O(1) \end{aligned} \tag{9}$$

8. (TA1) $\texttt{treeAdd}(Nil, l) = Nil$
   (TA2) $\texttt{treeAdd}(t, Empty) = t$
   (TA3) $\texttt{treeAdd}(Node(e, l, r), Cons(x, xs)) = Node(e + x, \texttt{treeAdd}(l, xs), \texttt{treeAdd}(r, xs))$

9. **Caz de bază** ($m = MEmpty$):
   $\texttt{get}(\texttt{update}(k, v, MEmpty), k) \overset{U1}{=} \texttt{get}(Insert(k, v, MEmpty), k) \overset{G1}{=} v$
   Pas inductiv Presupunem $\texttt{get}(\texttt{update}(k, v, m), k) = v$.
   Vrem să arătăm că $\texttt{get}(\texttt{update}(k, v, Insert(k', v', m)), k) = v$
   Din $U2$ avem două cazuri:
   $k = k'$: $\texttt{get}(Insert(k, v, m), k) \overset{G1}{=} v$
   $k \neq k'$: $\texttt{get}(Insert(k', v', \texttt{update}(k, v, m)), k) \overset{G1}{=} \texttt{get}(\texttt{update}(k, v, m), k) \overset{II}{=} v$